

# **MANUEL D'UTILISATION**

Programmateur horaire ESP32 – Multi-relais – Interface Web – OTA

Version V2

Manuel utilisateur – édition détaillée

# Sommaire

1. Présentation du système
2. Configuration matérielle
3. Première mise en service
4. Utilisation de l'interface Web
5. Programmation horaire
6. Modes AUTO et MANUEL
7. Boutons physiques
8. Wi-Fi et mDNS
9. Heure et programmation
10. Mémoire NVS et conservation des réglages
11. OTA – mise à jour du firmware
12. Commandes Web principales
13. Authentification
14. Dépannage
15. Procédure de test complète
16. Précautions électriques
17. Checklist utilisateur

# 1. Présentation du système

## Le projet

Ce système transforme un ESP32 en programmeur horaire connecté capable de commander plusieurs relais indépendants. La programmation se fait depuis une interface Web accessible avec un ordinateur, une tablette ou un smartphone.

## Fonctions principales

Le système permet de programmer plusieurs plages horaires par relais, de fonctionner en mode AUTO ou MANUEL, de forcer temporairement un relais, de mémoriser les réglages dans la mémoire NVS de l'ESP32, d'afficher les informations sur un écran OLED, de se reconnecter au Wi-Fi et de recevoir des mises à jour du firmware par OTA.

## Principe général

En mode AUTO, l'ESP32 compare en permanence l'heure courante avec les plages enregistrées et commande chaque relais. En mode MANUEL, l'horaire est ignoré pour le relais concerné et son état est conservé jusqu'au retour en AUTO.

# 2. Configuration matérielle

## Relais

La configuration actuelle prévoit quatre relais : relais 1 sur GPIO32, relais 2 sur GPIO33, relais 3 sur GPIO25 et relais 4 sur GPIO26. La logique actuelle est active HIGH. Si votre carte relais est active LOW, le paramètre du relais concerné doit être modifié dans le programme.

## Boutons physiques

Les boutons associés utilisent GPIO14, GPIO16, GPIO17 et GPIO18. Ils sont configurés avec INPUT\_PULLUP et doivent être raccordés entre la GPIO et GND. Un appui est détecté avec anti-rebond.

## OLED

L'écran OLED SSD1306 128×64 utilise I2C : SDA sur GPIO21, SCL sur GPIO22 et adresse I2C 0x3C. Il affiche l'heure, le réseau, l'adresse IP, l'état des relais et les informations de programmation.

# 3. Première mise en service

## Avant la mise sous tension

Vérifier le câblage, l'alimentation, les masses communes, la compatibilité 3,3 V des entrées des relais et l'absence de tension secteur sur les parties basse tension.

## Configuration Wi-Fi

Le fichier `arduino_secrets.h` doit contenir les identifiants Wi-Fi nécessaires et le mot de passe OTA. Ce fichier doit rester privé.

## Premier téléversement

Effectuer le premier téléversement par USB depuis Arduino IDE. Ouvrir ensuite le moniteur série à 115200 bauds pour vérifier la connexion Wi-Fi, l'adresse IP et la synchronisation de l'heure.

## Accès à la page Web

Utiliser l'adresse IP affichée par l'ESP32. Lorsque le mDNS fonctionne sur le réseau local, l'adresse `http://richardv.local` peut également être utilisée.

## 4. Utilisation de l'interface Web

### Ouverture

Ouvrir `http://richardv.local` ou `http://adresse_IP_de_l_ESP32` depuis un appareil connecté au même réseau local.

### État d'un relais

La page présente chaque programmeur avec son état, son mode AUTO/MANUEL et ses plages horaires. Les informations sont actualisées périodiquement.

### Modifier une programmation

Sélectionner le relais concerné, modifier les heures de début et de fin, puis enregistrer. Dans cette version, la route de sauvegarde des plages ne demande pas d'authentification.

### 24:00

24:00 représente minuit. Cette valeur est traitée spécifiquement car les champs HTML de type time ne prennent pas tous en charge 24:00 de manière identique. Une plage 06:00–24:00 signifie que le relais est actif de 06:00 jusqu'à minuit.

### Supprimer une plage

Une plage peut être supprimée depuis l'éditeur. Une programmation vide signifie qu'aucune plage n'est définie pour le relais.

## 5. Programmation horaire

### Format

Les horaires sont exprimés HH:MM-HH:MM. Plusieurs plages sont séparées par une virgule. Exemple : 06:30-08:00,12:00-13:30,18:00-22:00.

### Plage traversant minuit

Une plage 23:00-02:00 est valide. Elle couvre 23:00–24:00 puis 00:00–02:00. Il n'est pas nécessaire de la couper en deux plages.

### Chevauchement

Deux plages qui se chevauchent sont traitées comme une période continue. Exemple : 06:00-10:00 et 08:00-12:00 maintiennent le relais actif de 06:00 à 12:00.

### Validation

Avant sauvegarde, le programme contrôle le format, les heures, les minutes, le nombre maximal de plages et les plages de durée nulle. En cas d'erreur, l'ancienne configuration reste conservée.

### Nombre maximal

La configuration actuelle autorise six plages par relais. La valeur est définie par `MAX_PLAGES` dans le programme.

## 6. Modes AUTO et MANUEL

### AUTO

Le relais suit exclusivement la programmation horaire. Lorsque l'heure entre dans une plage active, le relais est activé ; lorsqu'elle en sort, il est désactivé.

### MANUEL

Le relais est commandé manuellement et ses horaires sont ignorés. Cette fonction est utile pour tester une charge ou maintenir volontairement un relais dans un état donné.

## **Forçage**

La commande de forçage passe le relais en MANUEL et applique immédiatement le nouvel état physique.

## **Retour en AUTO**

La commande Reset AUTO remet les relais en automatique et réapplique immédiatement leur programmation.

# **7. Boutons physiques**

## **Fonctionnement**

Un appui sur le bouton associé à un relais inverse son état, passe le relais en MANUEL et sauvegarde ce nouvel état.

## **Anti-rebond**

Un filtrage logiciel de 40 ms évite les déclenchements multiples dus aux rebonds mécaniques du bouton.

## **Test**

Tester chaque bouton séparément. Si un bouton ne répond pas, vérifier la GPIO, le raccordement à GND et le mode INPUT\_PULLUP.

# **8. Wi-Fi et mDNS**

## **Connexion**

Au démarrage, l'ESP32 recherche les réseaux connus et tente de sélectionner un réseau disponible offrant un bon niveau RSSI. La reconnexion est réalisée sans bloquer durablement la boucle principale.

## **RSSI**

La puissance du signal est affichée en dBm. Une valeur moins négative correspond généralement à un signal plus fort : -45 dBm est plus fort que -75 dBm.

## **mDNS**

Lorsque le service mDNS est correctement annoncé et pris en charge par le client, l'ESP32 est accessible avec <http://richardv.local>. Si le nom ne fonctionne pas, utiliser l'adresse IP.

## **Changement de réseau**

Le programme peut rechercher périodiquement un autre réseau connu offrant un meilleur signal. Une marge RSSI évite les basculements trop fréquents.

# **9. Heure et programmation**

## **NTP**

L'ESP32 synchronise son horloge avec des serveurs NTP. Le fuseau configuré correspond à la France avec prise en compte de l'heure d'été et de l'heure d'hiver.

## **Après redémarrage**

Après la synchronisation, la programmation est appliquée immédiatement afin que les relais retrouvent leur état correspondant à l'heure courante.

## **Échec NTP**

Le démarrage continue même si la synchronisation NTP échoue après les tentatives prévues. Une heure correcte reste toutefois indispensable pour une programmation fiable.

## 10. Mémoire NVS et conservation des réglages

### Ce qui est mémorisé

Les plages horaires, les modes et les états nécessaires sont sauvegardés dans la mémoire NVS de l'ESP32.

### OTA sans effacement

Une simple recompilation ou mise à jour OTA ne provoque plus d'effacement automatique des réglages. La date/heure de compilation `FIRMWARE_BUILD` sert au diagnostic, pas à décider d'effacer la configuration.

### CONFIG\_VERSION

La variable `CONFIG_VERSION` identifie la structure des données NVS. Elle ne doit être augmentée que lorsqu'une modification du format de stockage rend l'ancienne configuration incompatible.

### Attention

Modifier `CONFIG_VERSION` peut entraîner une réinitialisation de la configuration. Les horaires utilisateur doivent alors être notés ou sauvegardés avant la mise à jour.

## 11. OTA – mise à jour du firmware

### Principe

Après une première installation USB, le firmware peut être mis à jour par Wi-Fi si l'environnement Arduino IDE détecte le port réseau de l'ESP32.

### Procédure

Compiler le programme, sélectionner le port réseau de l'ESP32 dans Arduino IDE, lancer le téléversement et attendre le redémarrage complet. Ne pas couper l'alimentation pendant l'opération.

### Vérification

Après redémarrage, vérifier `FIRMWARE_BUILD` dans les informations système et vérifier que les plages horaires sont toujours présentes.

## 12. Commandes Web principales

### GET /

Affiche la page Web.

### GET /get-config

Retourne la configuration nécessaire à la construction de l'interface.

### GET /get-data

Retourne l'état courant, les plages, les modes, l'heure et les informations utiles.

### GET /get-info

Retourne les informations système, réseau et firmware.

### GET /toggle-mode?id=1

Bascule le relais concerné entre AUTO et MANUEL. Commande protégée.

### **GET /force-state?id=1**

Force l'état du relais et passe en MANUEL. Commande protégée.

### **POST /save?id=1**

Sauvegarde les plages horaires. Dans cette version, cette commande ne demande pas d'authentification.

### **GET /reset-auto**

Remet les relais en AUTO et applique la programmation. Commande protégée.

## **13. Authentification**

### **Plages horaires**

La modification des plages via /save est volontairement sans authentification dans cette version, conformément à la configuration demandée.

### **Commandes sensibles**

Les commandes de forçage, de changement AUTO/MANUEL et de retour global en AUTO restent protégées par HTTP Basic Authentication.

### **Identifiants**

L'utilisateur est admin. Le mot de passe est fourni par SECRET\_OTA\_PASSWORD dans arduino\_secrets.h.

### **Sécurité**

Le serveur fonctionne en HTTP et non HTTPS. L'authentification ne doit donc pas être considérée comme une protection cryptographique forte. Ne pas exposer directement l'ESP32 à Internet.

## **14. Dépannage**

### **richardv.local ne fonctionne pas**

Tester d'abord l'adresse IP. Si l'IP fonctionne, le problème est probablement lié au mDNS, au navigateur, au réseau invité, à l'isolation Wi-Fi ou à un VPN. Vérifier que le smartphone et l'ESP32 sont sur le même réseau local.

### **Le relais fonctionne à l'envers**

Vérifier relayActiveHigh. Une carte relais active LOW doit utiliser false pour le relais concerné.

### **Les horaires ont disparu**

Vérifier CONFIG\_VERSION. Une augmentation volontaire de cette version peut provoquer une réinitialisation NVS. Une simple OTA normale ne doit plus effacer les réglages.

### **L'heure est incorrecte**

Vérifier la connexion Internet nécessaire au NTP, le fuseau horaire et la disponibilité des serveurs NTP.

### **Un bouton déclenche plusieurs fois**

Vérifier le câblage et le filtrage anti-rebond. La valeur actuelle est 40 ms.

### **Une plage est refusée**

Vérifier le format HH:MM-HH:MM, les limites des heures et minutes, le nombre maximal de plages et l'absence de durée nulle.

### **OTA indisponible**

Vérifier que l'ESP32 est connecté au Wi-Fi, que le port réseau apparaît dans Arduino IDE, que le mot de passe OTA est correct et que le pare-feu du PC n'empêche pas la découverte.

## 15. Procédure de test complète

### Test 1 – Wi-Fi

Vérifier SSID, adresse IP, RSSI et éventuellement richardv.local.

### Test 2 – OLED

Vérifier l'affichage de l'heure, du réseau et des relais.

### Test 3 – Relais

Tester chaque relais en MANUEL avant de connecter une charge définitive.

### Test 4 – Boutons

Tester chaque bouton et vérifier le passage en MANUEL.

### Test 5 – Horaire

Créer une plage courte et vérifier ON/OFF automatiquement.

### Test 6 – Minuit

Tester une plage du type 23:55-00:05.

### Test 7 – NVS

Redémarrer l'ESP32 et vérifier que la programmation est conservée.

### Test 8 – OTA

Mettre à jour le firmware puis vérifier que les horaires sont toujours présents.

## 16. Précautions électriques

### Sécurité

Si les relais commandent du 230 V AC, des moteurs, du chauffage ou toute autre charge dangereuse, le câblage doit être réalisé avec les protections, l'isolation, les distances d'isolement et les composants adaptés. Ne jamais manipuler une partie secteur sous tension.

### Alimentation

L'alimentation de l'ESP32 et celle des relais doivent être dimensionnées correctement. Vérifier les masses et les niveaux logiques avant mise sous tension.

## 17. Checklist utilisateur

### Avant utilisation

- Wi-Fi configuré
- Adresse IP connue
- OLED fonctionnel
- Relais testés
- Boutons testés

- Heure correcte
- Horaires enregistrés

## Après modification

- Horaires validés
- Relais dans le mode souhaité
- NVS conservée après redémarrage

## Après OTA

- Nouveau firmware détecté
- Horaires conservés
- Heure correcte
- Relais dans l'état attendu

## Annexe A – Tableau des GPIO

| Élément  | Fonction        | GPIO   | Remarque                |
|----------|-----------------|--------|-------------------------|
| Relais 1 | Sortie relais   | GPIO32 | Actif HIGH actuellement |
| Relais 2 | Sortie relais   | GPIO33 | Actif HIGH actuellement |
| Relais 3 | Sortie relais   | GPIO25 | Actif HIGH actuellement |
| Relais 4 | Sortie relais   | GPIO26 | Actif HIGH actuellement |
| BP1      | Bouton relais 1 | GPIO14 | INPUT_PULLUP            |
| BP2      | Bouton relais 2 | GPIO16 | INPUT_PULLUP            |
| BP3      | Bouton relais 3 | GPIO17 | INPUT_PULLUP            |
| BP4      | Bouton relais 4 | GPIO18 | INPUT_PULLUP            |
| OLED SDA | I2C             | GPIO21 | SSD1306                 |
| OLED SCL | I2C             | GPIO22 | SSD1306                 |

## Annexe B – Paramètres principaux

| Paramètre          | Valeur        | Fonction                            |
|--------------------|---------------|-------------------------------------|
| MAX_PLAGES         | 6             | Nombre maximal de plages par relais |
| CONFIG_VERSION     | 2             | Version de la structure NVS         |
| BP_DEBOUNCE_MS     | 40 ms         | Anti-rebond des boutons             |
| RSSI_SWITCH_MARGIN | 8 dBm         | Marge pour changement de réseau     |
| Nom mDNS           | richardv      | Adresse richardv.local              |
| Port Web           | 80            | HTTP                                |
| OLED               | 128×64 / 0x3C | SSD1306 I2C                         |